

Mastering Continuous Q-Learning & Soft Actor-Critic

Complete Mathematical & Algorithmic Foundations: Target Networks, Polyak Averaging, Jensen's Overestimation Bias Proof, Clipped Twin-Q, Maximum Entropy RL Derivation, Dueling Architectures, and Production SAC

Instructor: Prof. Sergey Levine **Curriculum:** Berkeley CS285 + Haarnoja & Levine (UC Berkeley) (SAC) + Achiam (Spinning Up)

Scope: General Continuous Off-Policy Control & Robotics Slicing

0. The Executive Mental Map: Why Does Lecture 8 Exist?

In Lecture 6, we studied on-policy Actor-Critic methods (like A2C and PPO). On-policy algorithms collect a batch of data, execute gradient updates, and **immediately discard the entire dataset**.

The Sample Inefficiency Dilemma: Discarding data is acceptable in GPU simulators where data is virtually free. But on a physical robot or in expensive high-fidelity finite-element simulations, throwing away transitions (s, a, r, s') after one glance is unacceptable. We need **Off-Policy algorithms** that store millions of historical interactions in an **Experience Replay Buffer** and reuse them thousands of times.

Lecture 8 solves the off-policy puzzle for continuous control: We examine why naive deep Q-learning explodes, how Target Networks and Double Q-learning stabilize optimization, and how **Soft Actor-Critic (SAC)** blends Q-learning with Maximum Entropy to create the gold-standard off-policy robotics algorithm.

1. Target Networks

Polyak Averaging
Freezing Moving Targets

2. Overestimation

Jensen's Inequality
Clipped Twin-Q Solution

3. Continuous Max

Intractable $\arg\max_a$
Actor Maximizer (DDPG/SAC)

4. Max Entropy RL

Reward + $\alpha H(\pi)$
Anti-Freezing Exploration

5. The Deadly Triad

Bootstrapping +
Buffer + Function Approx

Part 1: Stabilizing Q-Learning — Target Networks & Polyak Averaging

(Slides 1–25) In exact tabular Q-learning, the Bellman optimality update is mathematically guaranteed to converge. But when we replace the table with a Deep Neural Network $Q_\theta(s, a)$, training frequently explodes. Why?

1.1 The "Chasing Your Own Tail" Problem

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q_\theta(s, a) - \left[r + \gamma \max_{a'} Q_\theta(s', a') \right] \right)^2 \right]$$

Notice that network parameters θ appear in **both** the prediction $Q_\theta(s, a)$ and the target $r + \gamma \max_{a'} Q_\theta(s', a')$. Every gradient step taken to update the prediction simultaneously shifts the target itself! To break this destabilizing feedback loop, we maintain a secondary set of weights $\bar{\theta}$ called the **Target Network**:

$$y_t = r_t + \gamma \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a')$$



1.2 Polyak Soft Updates vs. Hard Periodic Copies

Instead of hard-copying weights every 10,000 steps (as in original Atari DQN), continuous control algorithms smoothly blend target weights after every training step:

$$\bar{\theta} \leftarrow \tau \theta + (1 - \tau) \bar{\theta} \quad (\text{with } \tau = 0.005)$$

This corresponds to an exponential decay half-life of:

$$t_{\text{half}} = \frac{\ln(2)}{\tau} = \frac{0.693}{0.005} \approx 138 \text{ gradient steps}$$

This prevents abrupt target discontinuities, ensuring smooth impedance gain transitions on physical robotic hardware.

Part 2: Overestimation Bias & Clipped Double Q-Learning

(Slides 26–42) A profound mathematical flaw exists in standard Q-learning: **the maximization step systematically overestimates value functions!**

2.1 Mathematical Origin of Overestimation (Jensen's Inequality)

THEOREM: MAXIMIZATION OVER RANDOM VARIABLES INDUCES POSITIVE BIAS

Let $X_1, X_2, \dots, X_m$ be independent random variables representing noisy value estimates of true values $\mu_1, \dots, \mu_m$, where $X_i = \mu_i + \epsilon_i$ with zero-mean noise $\mathbb{E}[\epsilon_i] = 0$. Because the maximum function $f(x) = \max_i x_i$ is strictly convex, by **Jensen's Inequality**:

$$\mathbb{E} \left[\max_i X_i \right] \geq \max_i \mathbb{E}[X_i] = \max_i \mu_i$$

Numerical Demonstration: Suppose two actions have identical true value $Q(s, a_1) = Q(s, a_2) = 0$. Due to function approximation error, the neural network predicts noisy estimates $X_1 \sim \mathcal{N}(0, 1)$ and $X_2 \sim \mathcal{N}(0, 1)$.

$$\mathbb{E}[\max(X_1, X_2)] = \frac{1}{\sqrt{\pi}} \approx +0.564 > 0$$

Every single Bellman update injects positive error $+0.564$. Bootstrapping propagates this error forward exponentially: $V(s) \rightarrow V(s) + \gamma\Delta + \gamma^2\Delta + \dots \Rightarrow$ Q-values explode to $+10,000$, destroying policy gradients!

2.2 The Clipped Twin-Q Solution (Fujimoto et al., 2018 / Haarnoja et al., SAC)

To solve overestimation, maintain **two completely independent Critic networks** (Q_{ϕ_1} and Q_{ϕ_2}) with separate initializations. When computing the Bellman target, evaluate both and take the **minimum**:

$$y_t = r_t + \gamma \min \left(Q_{\bar{\phi}_1}(s_{t+1}, \tilde{a}_{t+1}), Q_{\bar{\phi}_2}(s_{t+1}, \tilde{a}_{t+1}) \right)$$

Taking the minimum injects controlled pessimism, completely neutralizing overestimation bias without requiring slow optimization.

Part 3: Continuous Actions & The Intractable Max

(Slides 43–58) In discrete environments (Atari), finding $\max_a Q(s, a)$ requires evaluating the network across 4 actions. In continuous robotics with $a \in \mathbb{R}^6$ (joint torques), finding the continuous global maximum is a non-convex optimization problem that cannot be solved in real-time.

Approach	Mechanism	Pros & Cons for Robotics
Stochastic Optimization (CEM / CMA-ES)	Sample 1,000 random actions, evaluate $Q(s, a)$, fit Gaussian to top 10%, repeat 3 times.	Extremely slow in the inner loop; fails beyond 10–20 action dimensions; unacceptable for real-time 60 Hz execution.
Analytical Q-Functions (NAF)	Constrain $Q(s, a)$ to be quadratic in a : $Q(s, a) = V(s) - \frac{1}{2}(a - \mu)^T P(a - \mu)$.	Maximum is analytically $\mu(s)$, but restricts the Critic to simple parabolic shapes; cannot capture contact bifurcations.
Learned Actor Maximizer (DDPG / SAC)	Train a separate neural policy $\pi_\theta(s)$ whose explicit objective is to output the action that maximizes $Q(s, a)$.	The Winning Approach: Computing the maximum requires only a single forward pass through the Actor network $\pi_\theta(s)$!

Part 4: Soft Actor-Critic (SAC) & Maximum Entropy RL

(Slides 59–75 & Spinning Up ch19) Soft Actor-Critic modifies the standard objective to maximize both **expected reward** AND **policy entropy** $\mathcal{H}(\pi)$:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

4.1 The "Anti-Freezing" Phenomenon in Soft Tissue Manipulation

Touching delicate biological tissue risks incurring severe negative penalties for crushing or excessive force ($F_z > 8 \text{ N}$). Under standard RL, the policy frequently gets trapped in a local minimum where the blade **hovers stationary 1 mm above the tissue** to collect zero penalties.

Under Maximum Entropy SAC: A frozen stationary blade has zero entropy ($\mathcal{H} = 0$), which incurs a massive entropy penalty! The robot is compelled to keep vibrating and testing sawing actions, discovering that light lateral sawing punctures the skin cleanly without crushing.

4.2 Automatic Entropy Temperature Tuning (α)

Rather than keeping temperature α fixed, SAC formulates temperature optimization as a dual constrained optimization problem targeting a heuristic $\bar{\mathcal{H}} = -\dim(\mathcal{A})$:

$$\mathcal{L}(\alpha) = \mathbb{E}_{a \sim \pi} \left[-\alpha (\log \pi(a | s) + \bar{\mathcal{H}}) \right]$$

Part 5: Sutton's Deadly Triad & Production SAC Implementation

(Slides 76–88) Richard Sutton proved that divergence in reinforcement learning arises whenever three algorithmic elements are combined simultaneously:

1. **Function Approximation:** Deep Neural Networks estimating continuous value landscapes.
2. **Bootstrapping:** Updating value estimates based on other value estimates: $y = r + \gamma Q(s', a')$.
3. **Off-Policy Learning:** Training on historical replay buffer data $\mathcal{D}$ generated by older policies.

SAC survives the Deadly Triad through four anchors: 1) Target networks, 2) Clipped Twin-Q pessimism, 3) Polyak averaging, and 4) Entropy regularization.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def compute_sac_losses(actor, q1, q2, target_q1, target_q2, log_alpha, batch, gamma=0.99):
    states, actions, rewards, next_states, dones = batch
    alpha = log_alpha.exp()

    # 1. CRITIC LOSS: Bellman Target using Target Twin-Q
    with torch.no_grad():
        next_actions, next_log_pi = actor.sample(next_states)
        q1_target = target_q1(next_states, next_actions)
        q2_target = target_q2(next_states, next_actions)
        min_next_q = torch.min(q1_target, q2_target) - alpha * next_log_pi
        y = rewards + gamma * (1.0 - dones) * min_next_q

    q1_loss = F.mse_loss(q1(states, actions), y)
    q2_loss = F.mse_loss(q2(states, actions), y)
    critic_loss = q1_loss + q2_loss

    # 2. ACTOR LOSS: Reparameterized Gradient
    new_actions, log_pi = actor.sample(states)
    min_q = torch.min(q1(states, new_actions), q2(states, new_actions))
    actor_loss = (alpha.detach() * log_pi - min_q).mean()

    # 3. TEMPERATURE LOSS: Target entropy heuristic -dim(A)
    target_entropy = -float(actions.shape[-1])
    alpha_loss = -(log_alpha * (log_pi + target_entropy).detach()).mean()

    return critic_loss, actor_loss, alpha_loss
```

Part 6: Interactive Tablet Self-Test Quiz

Question 1: Why does Clipped Twin-Q use the minimum of two target Q-networks rather than their average?

Answer: Taking the average does not prevent overestimation bias; if both networks have positive noise, their average remains positively biased. Taking the minimum $\min(Q_1, Q_2)$ introduces a mild, controlled underestimation bias that acts as a conservative safety margin, preventing runaway value explosion during contact manipulation.

Question 2: In SAC, why is the target entropy heuristic set to $-\dim(\mathcal{A})$?

Answer: A standard Gaussian distribution $\mathcal{N}(0, I)$ has entropy $\frac{d}{2}(1 + \ln(2\pi))$. Haarnoja and Levine found empirically that setting target entropy $\bar{\mathcal{H}} = -d$ (where $d = \dim(\mathcal{A})$) scales linearly with action space dimensionality, ensuring sufficient exploration across all 6 motor degrees of freedom without overwhelming the task reward.

Question 3: How does Polyak averaging ($\tau = 0.005$) stabilize training on physical robots?

Answer: Polyak averaging smoothly blends target network weights with an exponential half-life of ~ 138 steps ($\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta}$). This prevents the sudden value step-discontinuities that occur during periodic hard target copies, eliminating violent torque spikes at robot arm joints.

Part 7: Thesis Defense Master Cheatsheet (Lecture 8 Focus)

Q1: "WHY DOES STANDARD DEEP Q-NETWORKS (DQN) FAIL IN CONTINUOUS ROBOTIC MANIPULATION?"

Answer: "DQN selects actions via $\arg \max_a Q(s, a)$. In discrete games with 4 buttons, this requires 4 forward evaluations. In continuous multi-joint manipulation where the action space is $\mathbb{R}^6$ or $\mathbb{R}^{14}$, finding the continuous global maximum of an arbitrary neural network is an intractable non-convex optimization problem that cannot be solved within our 60 Hz control loop. Algorithms like DDPG and SAC solve this by training a dedicated Actor network to approximate the maximizer directly."

Q2: "WHY IS SOFT ACTOR-CRITIC (SAC) CONSIDERED THE GOLD STANDARD FOR SAMPLE-EFFICIENT REAL-WORLD ROBOT LEARNING?"

Answer: "SAC is an off-policy algorithm that reuses historical interactions from an experience replay buffer, requiring orders of magnitude fewer physical environment samples than on-policy PPO. By integrating Maximum Entropy RL with Clipped Twin-Q targets and soft Polyak averaging, SAC maintains active exploration, avoids premature policy freezing, and guarantees robust convergence in contact-rich physical tasks."